

Competitive Programming

Section 5 Editorial: Sorting as a tool

Competitive Programming Course

2026-07-04

Table of contents

1	How to read this editorial	3
2	Problem A: Minimum Adjacent Gap	4
2.1	Restatement	4
2.2	Important constraints and traps	4
2.3	Observations	4
2.4	Sample walk-through	4
2.5	Algorithm	5
2.6	Correctness	5
2.7	Complexity	5
2.8	C++ solution	5
2.9	Python solution	6
2.10	Common wrong answers	7
2.11	Tests that matter	7
3	Problem B: Rank Table	8
3.1	Restatement	8
3.2	Important constraints and traps	8
3.3	Observations	8
3.4	Sample walk-through	8
3.5	Algorithm	9
3.6	Correctness	9
3.7	Complexity	9
3.8	C++ solution	9
3.9	Python solution	10
3.10	Common wrong answers	11
3.11	Tests that matter	11
4	Problem C: Merge Intervals	12
4.1	Restatement	12
4.2	Important constraints and traps	12
4.3	Observations	12
4.4	Sample walk-through	12
4.5	Algorithm	13
4.6	Correctness	13
4.7	Complexity	13
4.8	C++ solution	14
4.9	Python solution	14
4.10	Common wrong answers	15
4.11	Tests that matter	15
5	Stress testing strategy	16
6	What to carry forward	17

1 How to read this editorial

Sorting problems are often solved by a short call to `sort`, but the important work is deciding what the sorted order proves.

For each problem, look for:

- what information becomes adjacent or ordered after sorting;
- what exact key or comparator is required;
- what invariant the left-to-right scan maintains.

2 Problem A: Minimum Adjacent Gap

2.1 Restatement

Given n integers, find the minimum absolute difference between two distinct array elements.

2.2 Important constraints and traps

The total n is at most 200000, so sorting is acceptable. Checking all pairs is $O(n^2)$ and too slow.

Duplicate values are allowed. If any value appears twice, the answer is 0.

The input may contain negative numbers, but after sorting, adjacent differences are nonnegative.

2.3 Observations

After sorting the values:

```
b1 <= b2 <= ... <= bn
```

the closest pair must be adjacent. Therefore it is enough to compute:

```
min(b[i] - b[i-1]) for i = 1..n-1
```

2.4 Sample walk-through

Values:

```
8 1 4 10 6
```

Sorted:

```
1 4 6 8 10
```

Adjacent gaps:

pair	gap
1, 4	3

pair	gap
4, 6	2
6, 8	2
8, 10	2

The answer is 2.

2.5 Algorithm

```

sort values
best = values[1] - values[0]
for i from 2 to n - 1:
    best = min(best, values[i] - values[i - 1])
print best

```

2.6 Correctness

Let the sorted values be $b_1 \leq b_2 \leq \dots \leq b_n$.

Consider any pair b_i, b_j with $i < j$. If $j = i + 1$, the pair is adjacent and is checked by the algorithm. If $j > i + 1$, then there is at least one adjacent gap inside the interval from i to j . The sum of those adjacent gaps is:

$$b_j - b_i.$$

Since all gaps are nonnegative, at least one adjacent gap in that interval is no larger than $b_j - b_i$. Therefore every non-adjacent pair has an adjacent pair with difference no larger. The global minimum is achieved by some adjacent pair.

The algorithm checks every adjacent pair and returns the smallest adjacent difference. By the argument above, this is the minimum over all pairs.

2.7 Complexity

Sorting takes $O(n \log n)$. The scan is $O(n)$. Memory is $O(1)$ besides the array, depending on the sorting implementation.

2.8 C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<int> values(n);
        for (int &value : values) {
            cin >> value;
        }
        sort(values.begin(), values.end());
        int best = values[1] - values[0];
        for (int i = 2; i < n; ++i) {
            best = min(best, values[i] - values[i - 1]);
        }
        cout << best << '\n';
    }
    return 0;
}

```

2.9 Python solution

```

import sys

def min_gap(values: list[int]) -> int:
    values.sort()
    return min(values[i] - values[i - 1] for i in range(1, len(values)))

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    idx += 1
    values = data[idx:idx + n]
    idx += n
    out.append(str(min_gap(values)))
sys.stdout.write("\n".join(out))

```

2.10 Common wrong answers

- Checking only the original adjacent elements before sorting.
- Forgetting duplicate values.
- Using absolute differences after sorting but accidentally subtracting in the wrong direction.
- Using an $O(n^2)$ pair loop.

2.11 Tests that matter

```
duplicates: answer 0  
two elements only  
negative and positive extremes  
already sorted input  
reverse sorted input
```

3 Problem B: Rank Table

3.1 Restatement

Sort contestants by:

1. higher score first;
2. lower penalty first;
3. lexicographically smaller name first.

Print the names in that order.

3.2 Important constraints and traps

Each key has its own direction. Score is descending; penalty and name are ascending.

The final name tie-breaker matters. Without it, different accepted-looking programs could print different orders.

3.3 Observations

This is a direct custom-key sorting problem.

In Python, use:

```
(-score, penalty, name)
```

In C++, write the three comparisons explicitly.

3.4 Sample walk-through

Rows:

```
alice 300 50
bob   400 90
carol 400 70
dave  300 40
```

Sorted key values:

name	key
carol	(-400, 70, carol)
bob	(-400, 90, bob)
dave	(-300, 40, dave)
alice	(-300, 50, alice)

Output:

```
carol bob dave alice
```

3.5 Algorithm

```
read rows
sort by score descending, penalty ascending, name ascending
print names
```

3.6 Correctness

The comparator compares two contestants exactly according to the statement's priority list. If their scores differ, the one with larger score is placed first. If scores are equal but penalties differ, the one with smaller penalty is placed first. If both score and penalty are equal, the lexicographically smaller name is placed first.

Sorting arranges the entire list so that no adjacent pair violates this order. Because the comparator encodes the required total ordering, the sorted list is exactly the required ranking.

3.7 Complexity

Sorting n contestants takes $O(n \log n)$ comparisons. Memory is $O(n)$ to store the rows.

3.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Row {
    string name;
    int score;
    int penalty;
};

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);

int T;
cin >> T;
while (T--) {
    int n;
    cin >> n;
    vector<Row> rows(n);
    for (auto &row : rows) {
        cin >> row.name >> row.score >> row.penalty;
    }
    sort(rows.begin(), rows.end(), [](const Row &a, const Row &b) {
        if (a.score != b.score) return a.score > b.score;
        if (a.penalty != b.penalty) return a.penalty < b.penalty;
        return a.name < b.name;
    });
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << rows[i].name;
    }
    cout << '\n';
}
return 0;
}

```

3.9 Python solution

```

import sys

data = sys.stdin.buffer.read().split()
t = int(data[0])
idx = 1
out = []
for _ in range(t):
    n = int(data[idx])
    idx += 1
    rows = []
    for _ in range(n):
        name = data[idx].decode()
        score = int(data[idx + 1])
        penalty = int(data[idx + 2])
        idx += 3
        rows.append((name, score, penalty))
    rows.sort(key=lambda row: (-row[1], row[2], row[0]))
    out.append(" ".join(row[0] for row in rows))
sys.stdout.write("\n".join(out))

```

3.10 Common wrong answers

- Sorting score ascending.
- Sorting penalty descending.
- Forgetting the name tie-breaker.
- Writing a C++ comparator with `>=`.
- Assuming stable sort will solve missing tie rules.

3.11 Tests that matter

```
all scores different
same score, different penalty
same score and penalty, different names
single contestant
many ties
```

4 Problem C: Merge Intervals

4.1 Restatement

Given closed integer intervals, merge every pair of intervals that overlaps or touches. Touching means the next interval starts at most one position after the current interval ends.

Print the merged intervals in increasing order.

4.2 Important constraints and traps

The total number of intervals is up to 200000, so sorting is acceptable.

The merge condition is:

```
next_l <= current_r + 1
```

because touching intervals merge. If you use only `next_l <= current_r`, you will fail cases such as `[1, 3]` and `[4, 5]`.

4.3 Observations

Sort intervals by left endpoint. Then scan left to right. The next interval can only affect the last merged interval:

- if it starts after the last merged interval ends plus one, it begins a new component;
- otherwise it extends the last merged interval's right endpoint if needed.

4.4 Sample walk-through

Input intervals:

```
[5,7], [1,3], [4,4], [10,12], [11,20]
```

After sorting:

```
[1,3], [4,4], [5,7], [10,12], [11,20]
```

Scan:

next interval	action	merged list
[1,3]	start	[1,3]
[4,4]	touches	[1,4]
[5,7]	touches	[1,7]
[10,12]	separate	[1,7] , [10,12]
[11,20]	overlaps	[1,7] , [10,20]

4.5 Algorithm

```

sort intervals by l, then r
merged = empty list
for interval [l, r]:
    if merged is empty or l > merged.last.r + 1:
        append [l, r]
    else:
        merged.last.r = max(merged.last.r, r)
print merged

```

4.6 Correctness

After sorting, intervals are processed in nondecreasing order of left endpoint. Maintain the invariant:

after processing the first i sorted intervals, **merged** is the correct merged representation of those intervals.

Initially no intervals have been processed and the empty merged list is correct.

Consider the next interval $[l, r]$. If **merged** is empty, adding it is correct. Otherwise let the last merged interval be $[L, R]$.

If $l > R + 1$, the new interval starts strictly after the last interval ends and does not even touch it. Since all earlier merged intervals end before $[L, R]$, the new interval cannot merge with any earlier interval either. Starting a new merged interval preserves correctness.

If $l \leq R + 1$, the new interval overlaps or touches $[L, R]$, so both belong to the same merged component. Their union is $[L, \max(R, r)]$. Replacing the last interval with this union preserves the correct merged representation.

Thus the invariant holds after every step. After all intervals are processed, **merged** is the correct answer.

4.7 Complexity

Sorting takes $O(n \log n)$. The scan is $O(n)$. Memory is $O(n)$ for the output list.

4.8 C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<pair<int, int>> intervals(n);
        for (auto &interval : intervals) {
            cin >> interval.first >> interval.second;
        }
        sort(intervals.begin(), intervals.end());

        vector<pair<int, int>> merged;
        for (auto [l, r] : intervals) {
            if (merged.empty() || l > merged.back().second + 1) {
                merged.push_back({l, r});
            } else {
                merged.back().second = max(merged.back().second, r);
            }
        }

        cout << merged.size() << '\n';
        for (auto [l, r] : merged) {
            cout << l << ' ' << r << '\n';
        }
    }
    return 0;
}
```

4.9 Python solution

```
import sys

def merge(intervals: list[tuple[int, int]]) -> list[tuple[int, int]]:
    intervals.sort()
    merged = []
    for l, r in intervals:
        if not merged or l > merged[-1][1] + 1:
            merged.append((l, r))
```

```

        else:
            old_l, old_r = merged[-1]
            merged[-1] = (old_l, max(old_r, r))
    return merged

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
idx = 1
out = []
for _ in range(t):
    n = data[idx]
    idx += 1
    intervals = []
    for _ in range(n):
        l = data[idx]
        r = data[idx + 1]
        idx += 2
        intervals.append((l, r))
    merged = merge(intervals)
    out.append(str(len(merged)))
    out.extend(f"{l} {r}" for l, r in merged)
sys.stdout.write("\n".join(out))

```

4.10 Common wrong answers

- Not sorting intervals first.
- Sorting by right endpoint and then using a start-point scan invariant.
- Forgetting to merge touching intervals.
- Updating the left endpoint of the current merged interval unnecessarily.
- Printing intervals in the original order.

4.11 Tests that matter

```

single interval
already sorted intervals
reverse sorted intervals
touching intervals
nested intervals
negative endpoints
separate intervals with one-cell gaps

```

5 Stress testing strategy

Sorting problems often have simple slow oracles.

For minimum gap, compare the adjacent-sort solution with the $O(n^2)$ all-pair minimum on small arrays.

For rank tables, generate random rows and sort using a trusted tuple key.

For interval merging, one slow option for small coordinate ranges is to mark all covered integer points, then reconstruct components. Another is to compare against an independently written merge implementation.

Random tests should include many ties and many duplicates. Sorting bugs often hide when all keys are distinct.

6 What to carry forward

Sorting is useful when it changes the geometry of the problem:

- closest values become adjacent;
- equal values become blocks;
- intervals become a left-to-right scan;
- tie rules become comparator keys.

Later sections will use sorting inside greedy algorithms, sweep lines, offline queries, coordinate compression, and graph construction.